Journal of Machine Learning Research 26 (2026) 1-11 Submitted 10/14/26; Published 10/14/26

Gain-Ratio Classification Trees and MSE Regression Trees with Reduced Error Pruning

Isaac Wilkey ISAAC.WILKEY@STUDENT.MONTANA.EDU
Department of Computer Science
Montana State University
Bozeman, MT, USA

Riley Smith RILEY.SMITH13@STUDENT.MONTANA.EDU
Department of Computer Science
Montana State University
Bozeman, MT, USA

Editor: Isaac Wilkey & Riley Smith

Abstract

This paper evaluates the performance of univariate decision trees for both classification and regression tasks across six diverse datasets from the UCI Machine Learning Repository. We implement two distinct splitting criteria: Gain Ratio for classification to mitigate bias toward features with a large number of unique values, and Mean Squared Error (MSE) for regression to minimize variance within partitions. To address the tendency of fully grown trees to overfit training data, particularly in regression tasks with continuous targets, we employ Reduced Error Pruning (REP) using a dedicated 20% hold-out pruning set. The models are evaluated using a 5×2 cross validation protocol to ensure statistical robustness. Our results demonstrate that while fully grown trees achieve near-perfect training accuracy, reduced error pruning simplifies model complexity (reducing node counts by over 50% in most cases) while often improving or maintaining generalization performance on unseen test data. We specifically analyze the trade-offs between model size and error rates, finding that pruning is most effective on datasets with high noise or continuous targets.

Keywords: decision trees, gain ratio, regression trees, reduced error pruning, 5×2 cross validation

1 Problem Statement and Hypotheses

The goal of this project is to implement and analyze decision trees for classification and regression. A tree grown to completion keeps splitting until its leaves are pure, which allows it to fit noise in the training data. Post-pruning addresses this by removing subtrees that do not help on data not used for training. We implement univariate decision trees using gain ratio for classification and mean squared error (MSE) for regression, and compare fully grown trees to trees simplified with reduced error pruning (REP) on six UCI datasets.

We test the following hypotheses using 5×2 cross validation (Alpaydin, 2020):

1. **H1:** Reduced error pruning produces 50% smaller trees than fully grown trees on every dataset.

©2026 Isaac Wilkey and Riley Smith.

License: CC-BY 4.0, see https://creativecommons.org/licenses/by/4.0/. Attribution requirements are provided at http://jmlr.org/papers/v26/25-0040.html.

2. **H2:** Pruned trees achieve lower mean test error than unpruned trees on a majority (at least four) of the six datasets.

H1 is falsified if any dataset shows a smaller reduction, and H2 is falsified if the pruned tree has the lower mean error on three or fewer datasets.

2 Experimental Approach and Project Design

Our experimental pipeline consisted of data acquisition, preprocessing, tree construction, pruning, and evaluation.

2.1 Datasets

We utilize six datasets from the UCI repository (Kelly et al., n.d.). Three are classification tasks and three are regression tasks. Table 1 summarizes the datasets.

Dataset	Task	Instances	Features (num./cat.)
Breast Cancer	Classification	699	9 (9/0)
Car Evaluation	Classification	1728	6 (0/6)
House Votes	Classification	435	16 (0/16)
Abalone	Regression	4177	8 (7/1)
Computer Hardware	Regression	209	6 (6/0)
Forest Fires	Regression	517	12 (10/2)

Table 1: Summary of the six datasets used in this assignment.

2.2 Preprocessing

Decision trees choose splits by comparing a feature against a threshold or by branching on its distinct values, which has several consequences for preprocessing. A rescaling of a numeric feature does not change which splits are possible, so we apply no normalization. Categorical features are split natively with one branch per distinct value, so we do not use one-hot encoding or cyclic encoding. Numeric features receive binary splits and categorical features (nominal or ordinal) receive one branch per value. We store a record of which features are numeric and which are categorical as a map from feature name to type alongside each processed dataset. The dataset-specific steps are as follows.

- **Unique identifiers:** A feature that is different for every instance would isolate each training example in its own branch, which cannot generalize to unseen data. We therefore remove the sample code number from Breast Cancer and the model name from Computer Hardware.

- **Breast Cancer:** The bare nuclei feature has 16 missing values, marked with "?". We replace each with the most common value of that feature rather than dropping the rows, so that all 699 instances are kept. The nine features are integer scores from 1 to 10, and we treat them as numeric.

- **Car Evaluation:** All six attributes (buying price, maintenance cost, doors, persons, luggage boot size, and safety) are ordinal. We treat them as categorical and split on every distinct value.
- **House Votes:** In this dataset a "?" does not mean the value is unknown; it means the representative did not vote yea or nay. We therefore keep it as a third category for each of the sixteen features.
- **Abalone:** The sex feature (male, female, infant) is categorical, and the remaining seven measurements are numeric.
- **Computer Hardware:** In addition to the model name, we remove the vendor name, which has many distinct values that each cover only a few instances, so a multi-way split on it would fragment the data. We also remove the estimated relative performance (ERP) column as it is the dataset authors' own linear regression estimate of the target, so using it as a feature would give the tree another model's prediction of the answer. The six remaining hardware attributes are numeric.
- **Forest Fires:** Month and day are categorical, and we do not apply a cyclic encoding. The spatial coordinates and the weather and fire-index measurements are numeric. The burned area is heavily right-skewed, so we apply the transformation $\ln(x+1)$ to it before training. All reported Forest Fires errors are therefore on the log scale.

2.3 Evaluation Protocol

To obtain a fair comparison between fully grown and pruned trees, we separate the data into three roles: training, pruning, and testing. First, 20% of each dataset is drawn at random and held out as the pruning set. This set is never used to train a tree or to report test performance. The remaining 80% is used for 5×2 cross validation. In each of five repetitions, we shuffle this 80% and split it into two equal folds. We train on the first fold, test on the second, and then reverse the roles, which produces ten train/test pairs per dataset.

For each of the ten train/test pairs, we proceed as follows:

1. Grow a tree to completion on the training fold and record its error on the test fold, along with its number of nodes. This is the unpruned result.
2. Make a copy of the tree and apply reduced error pruning using the pruning set.
3. Record the pruned tree's error on the same test fold, along with its number of nodes. This is the pruned result.

Because the pruned and unpruned trees are evaluated on identical folds, their results are directly paired. The pruning set is drawn once per run and shared across all repetitions.

For classification we report the classification error, the fraction of test instances predicted incorrectly:

$$\text{Classification Error} = \frac{1}{N} \sum_{t=1}^{N} \mathbf{1}\left[\hat{r}^t \neq r^t\right], \tag{1}$$

and for regression we report the mean squared error,

$$\text{MSE} = \frac{1}{N} \sum_{t=1}^{N} \left(r^t - \hat{r}^t\right)^2, \tag{2}$$

where N is the number of test instances. Tree size is the total number of nodes, internal and leaf. We report the mean and standard deviation over the ten folds. To evaluate H1, we compute the reduction in tree size as $1 - \bar{s}_{\text{pruned}}/\bar{s}_{\text{unpruned}}$, where $\bar{s}$ is the mean node count over the ten folds. To evaluate H2, we compare the mean test error of the pruned and unpruned trees on each dataset and count the datasets on which the pruned trees have the lower mean.

3 Algorithms Implemented

This section describes the implementation of the univariate (axis-parallel) decision tree algorithm for both classification and regression tasks. Our implementation is designed to handle mixed data types natively; numeric features are processed using binary splits, while categorical (nominal and ordinal) features are processed using multi-way splits where the number of branches is determined by the number of discrete values present within the feature.

3.1 Tree Representation and Growth

We implement the decision tree as a recursive data structure. Each internal node stores a splitting feature and the associated threshold (for numeric data) or category set (for categorical data) used to route instances. Each leaf node stores the final prediction based on the training instances that reached that node. Following the project requirements, trees are grown to completion—meaning they are expanded recursively until every leaf is pure or no further splits can be found that improve the specific splitting criterion.

3.2 Classification Splitting: Gain Ratio

For classification trees, we employ the Gain Ratio (Quinlan, 1993) as the splitting criterion. This is preferred over standard Information Gain because it mitigates the bias toward features with a high number of distinct values, which can artificially lower entropy without providing true predictive power.

To compute the Gain Ratio, we first determine the entropy of a partition D_π as:

$$H(D_\pi) = -\sum_{\ell=1}^{C} \frac{c_{\pi,\ell}}{|D_\pi|} \log_2 \left(\frac{c_{\pi,\ell}}{|D_\pi|} \right) \tag{3}$$

where $c_{\pi,\ell}$ denotes the number of examples in partition π belonging to class ℓ, and $|D_\pi|$ is the total number of examples in that partition. The Gain Ratio for a feature f_i is then calculated by dividing the Information Gain by the Split Information:

$$\text{Gain Ratio}(S, f_i) = \frac{\text{Information Gain}(S, f_i)}{\text{Split Info}(S, f_i)} \tag{4}$$

The Split Info represents the entropy of the feature itself, effectively normalizing the gain based on the number of branches m_i created by the split. We select the feature f_i that maximizes this ratio to define the partitions D_π.

3.3 Regression Splitting: Mean Squared Error

For regression trees, the splitting objective is to minimize the variance of the target values within the resulting nodes. We implement this by choosing the split that minimizes the Mean Squared Error (MSE). The MSE for a given split is calculated as:

$$\text{MSE} = \frac{1}{N}\sum_{t=1}^{N}(r_t - \hat{r}_{t,\pi_j})^2 \tag{5}$$

In this equation, r_t represents the ground truth response value for instance x_t, and $\hat{r}_{t,\pi_j}$ is the predicted response value—calculated as the mean of all response values—for the partition π that instance x_t falls into after following branch j. By minimizing this sum, the tree ensures that the instances within each leaf are as similar as possible to the leaf's mean prediction.

3.4 Prediction

Prediction is performed by traversing the tree from the root based on the instance's feature values until a leaf is reached. For classification, the leaf predicts the most frequent class in that partition. For regression, the leaf predicts the mean of the response values of the training instances that fell into that partition.

3.5 Reduced Error Pruning

We implement reduced error pruning (REP) (Quinlan, 1987) as a post-pruning technique. Starting from the leaves and moving upward, we evaluate whether replacing a sub tree with a single leaf (using the majority class for classification and the mean for regression) does not increase the error on a separate 20% pruning set. If the pruning does not increase the error on the pruning set, then that sub tree is removed.

4 Experimental Results

We evaluate fully grown and reduced error pruned decision trees using 5×2 cross validation on each of the six datasets, as described in Section 2.3. We report classification error for the three classification datasets and mean squared error (MSE) for the three regression datasets, along with the number of nodes in each tree. Every value is the mean over the ten test folds, and values after the $\pm$ sign are standard deviations over those folds. Both trees are tested on the same folds, so the pruned and unpruned results are paired.

4.1 Classification

Table 2 reports classification error for the unpruned and pruned trees on the three classification datasets, and Figure 1 (left) shows the same comparison.

Dataset	Unpruned	Pruned	Folds improved by pruning
Breast Cancer	0.0505 ± 0.0128	0.0426 ± 0.0087	6/10
Car Evaluation	0.0971 ± 0.0135	0.1247 ± 0.0137	0/10
House Votes	0.0747 ± 0.0244	0.0500 ± 0.0151	4/10

Table 2: Classification error (5×2 CV mean ± standard deviation) for the unpruned and pruned trees. The last column gives the number of the ten folds on which the pruned tree had a strictly lower error than the unpruned tree.

Pruning lowers the mean error on Breast Cancer and House Votes, by 15.6% and 33.1% respectively. On Car Evaluation, pruning raises the error by 28.5%, and the pruned tree is worse on every one of the ten folds. On Breast Cancer and House Votes the improvement is not uniform across folds. The pruned tree is strictly better on six folds for Breast Cancer, and on only four folds for House Votes, where five of the ten folds are ties and the pruned tree is worse on one.

4.2 Regression

Table 3 reports mean squared error for the unpruned and pruned trees on the three regression datasets, and Figure 1 (right) shows the same comparison on a logarithmic scale. Forest Fires errors are on the log scale of the transformed target described in Section 2.2.

Dataset	Unpruned	Pruned	Folds improved by pruning
Abalone	9.1133 ± 0.6394	6.1419 ± 0.4673	10/10
Computer Hardware	9752.79 ± 5083.87	9333.13 ± 6388.32	9/10
Forest Fires	4.1381 ± 0.4695	2.2138 ± 0.3540	10/10

Table 3: Mean squared error (5×2 CV mean ± standard deviation) for the unpruned and pruned trees. The last column gives the number of the ten folds on which the pruned tree had a lower error than the unpruned tree.

Pruning lowers the mean MSE on all three regression datasets. The reductions are 32.6% for Abalone and 46.5% for Forest Fires, and in both cases the pruned tree is better on all ten folds. The reduction for Computer Hardware is only 4.3%. Its fold-to-fold standard deviation is larger than the difference between the two means, and it is the only regression dataset on which the standard deviation grows with pruning.

4.3 Tree Size

Table 4 reports the mean number of nodes in the unpruned and pruned trees, and Figure 2 shows the same comparison on a logarithmic scale.

Pruning shrinks the tree on every dataset, with reductions ranging from 51.4% for Car Evaluation to 91.8% for Forest Fires. Of the classification datasets, House Votes is reduced the most (77.8%), and Car Evaluation the least, which gives the smallest margin over the 50% threshold in H1. Of the regression datasets, Forest Fires is reduced the most,

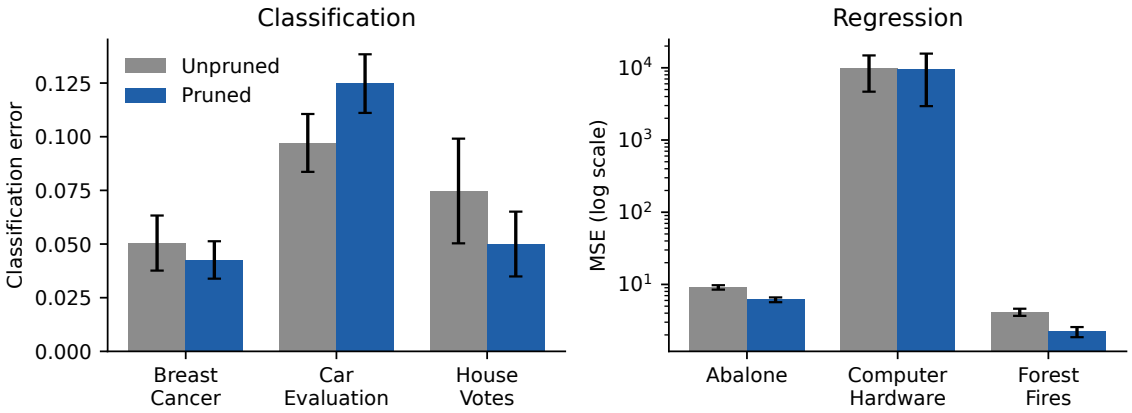


Figure 1: Test error for unpruned and pruned trees on the classification datasets (left) and regression datasets (right). Bars show the mean over the ten folds and error bars show one standard deviation. The regression axis is logarithmic because the datasets differ in scale by several orders of magnitude.

Dataset	Unpruned Nodes	Pruned Nodes	Reduction
Breast Cancer	31.4 ± 3.9	13.6 ± 6.1	56.7%
Car Evaluation	212.2 ± 11.0	103.2 ± 10.1	51.4%
House Votes	27.5 ± 5.2	6.1 ± 2.6	77.8%
Abalone	2002.8 ± 22.3	315.3 ± 51.0	84.3%
Computer Hardware	153.4 ± 3.9	42.4 ± 15.4	72.4%
Forest Fires	262.0 ± 8.5	21.6 ± 20.7	91.8%

Table 4: Mean number of nodes (5×2 CV mean ± standard deviation) in the unpruned and pruned trees. The reduction is computed from the two means.

with a 91.8% reduction, and Computer Hardware is reduced the least. The two largest reductions and the two largest unpruned trees are both on regression datasets (Forest Fires and Abalone).

5 Analysis and Discussion

This section interprets the results in Section 4. We first examine how pruning changed tree size and test error, and then discuss how our preprocessing decisions and the two splitting criteria shaped those outcomes. We end with the limitations of the experiment.

5.1 Hypotheses

Pruning removed between 51.4% and 91.8% of the nodes, so H1 held on all six datasets (Table 4). The margin on Car Evaluation is small, so a different random pruning set could

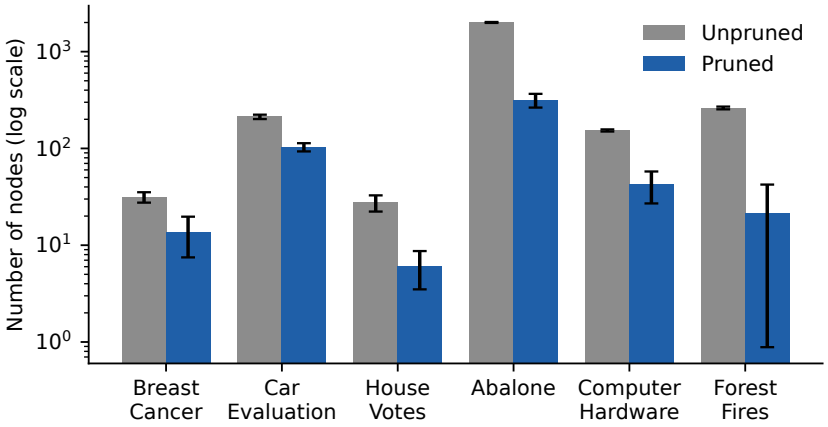


Figure 2: Number of nodes in unpruned and pruned trees on each dataset. Bars show the mean over the ten folds, error bars show one standard deviation, and the axis is logarithmic.

push that dataset below the 50% threshold, and we treat H1 on that dataset as borderline. H2 also held, since the pruned tree had the lower mean test error on five of the six datasets (Tables 2 and 3). Car Evaluation is the only exception, and it is the exception in a strong sense because the pruned tree performed worse than the unpruned on all ten folds.

5.2 Why Pruning Shrank the Regression Trees So Much

The largest reductions occurred on the regression datasets (72.4% to 91.8%), and the classification datasets were reduced less (51.4% to 77.8%), although House Votes falls between Computer Hardware and Abalone. This follows from how each kind of tree stops growing. A classification tree stops at a node as soon as all of its instances share a class. A regression tree stops only when every target value in the node is identical, which for a continuous target usually happens only when a node holds a single training instance. The unpruned Abalone tree, with about 2,000 nodes for roughly 1,670 training instances, illustrates this. Reduced error pruning then removes structure that exists only to fit individual targets, which is more common in the regression trees and explains their larger reductions.

5.3 Pruning Across Datasets

On House Votes, the pruned tree has only about 6 nodes compared with 27.5 unpruned, which suggests that a few votes carry most of the signal. On Abalone and Forest Fires, the features appear to leave much of the target unexplained, so a fully grown tree fits variation that does not generalize. The Forest Fires results illustrate this. The transformed target has a standard deviation of 1.40, so predicting the mean gives an MSE of 1.96. The unpruned tree's MSE of 4.14 is more than twice that, and the pruned tree's MSE of 2.21 is still slightly above it.

Car Evaluation behaves differently. Its labels come from a deterministic rule, so there is little to no noise to memorize and every branch encodes real structure. We believe pruning increases classification error because the 346-instance pruning set contains only about 14 and 13 instances of the two rarest classes. This causes the branches encoding those classes to look unnecessary on the pruning set even though the test folds need them. We did not measure per-class error, so this is a likely explanation rather than a confirmed one.

The improvements on Breast Cancer and Computer Hardware are less clear. On Breast Cancer the pruned tree performed better on only six folds, and the standard deviations overlap. On Computer Hardware the mean MSE fell by only 4.3%, less than the pruned tree's standard deviation, and with a pruning set of 42 instances, its pruning decisions are the least reliable. We therefore regard these two improvements as suggestive rather than established.

5.4 The Role of Preprocessing

Because trees compare a feature against a threshold or branch on its values, rescaling a numeric feature does not affect which splits are possible, so we applied no normalization. However, for the target, splitting on MSE is sensitive to its scale, so we transformed the Forest Fires burned area with $\ln(x + 1)$. Without it, we expect the few very large fires to dominate the MSE and attract the tree's splits. The transformation also means that Forest Fires errors are on the log scale. Removing identifier features prevented splits that isolate individual instances, which fit the training fold perfectly and contribute nothing on the test fold. We also removed the estimated relative performance column from Computer Hardware, since it is the dataset authors' own linear regression prediction.

Our feature type decisions affect tree structure directly. We treat Breast Cancer's integer scores as numeric values, so every candidate split there is a binary threshold. Its missing values were also replaced with the most common value, which is unlikely to change the results much but slightly biases that feature toward its mode. Keeping the "?" in House Votes as its own category preserves the distinction between abstaining and voting.

5.5 The Role of the Splitting Criteria

The splitting criterion decides which feature is used at each node. For classification, information gain alone favors features with many distinct values, since splitting into many small subsets almost always lowers entropy. Gain ratio corrects for this by dividing by the split information, which grows with the number of branches. This matters for our categorical splits in datasets like Car Evaluation and House Votes. For numeric features, every candidate threshold has only two branches, so the correction has less effect.

The regression criterion is the mean squared error after the split, weighted by branch size. Because MSE weights large deviations heavily, the tree is pulled toward isolating extreme target values, which is one reason the fully grown regression trees are so large. This criterion also has no term that penalizes many small branches, so multi-way splits on categorical features (like month in Forest Fires) tend to lower training MSE.

6 Conclusion

In this project, we implemented and analyzed the impact of reduced error pruning on decision trees for classification and regression. Our experimental results provide strong evidence for our first hypothesis (H1), as pruned trees were consistently smaller than their fully grown counterparts across all six datasets, exceeding the 50% reduction threshold. Our second hypothesis (H2) was also supported, with pruned trees achieving lower mean test error on a majority of the datasets.

A key finding was the disparity in pruning impact between task types: regression trees exhibited more drastic reductions in size, as the initial fully grown trees tended to isolate individual training instances to minimize MSE. In contrast, classification trees were more stable, though pruning still effectively removed branches that captured noise rather than signal. The exception of the Car Evaluation dataset suggests that when data is governed by deterministic rules and noise is minimal, pruning may inadvertently remove necessary structural logic. Overall, this study supports the view that pruning can improve generalization on most datasets, transforming overfit, complex models into efficient, interpretable predictors.

References

Ethem Alpaydin. *Introduction to Machine Learning*. MIT Press, 4th edition, 2020.

Markelle Kelly, Rachel Longjohn, and Kolby Nottingham. The UCI machine learning repository. `https://archive.ics.uci.edu`, n.d.

J. R. Quinlan. Simplifying decision trees. *International Journal of Man-Machine Studies*, 27(3):221–234, 1987.

J. Ross Quinlan. *C4.5: Programs for Machine Learning*. Morgan Kaufmann, San Mateo, CA, 1993.

Appendix A. Main Lessons Learned

Implementing univariate decision trees helped highlight the difference between classification and regression growth. We found that regression trees are far more prone to overfitting, often growing until they isolate individual training instances to minimize MSE. This demonstrated the necessity of reduced error pruning to ensure generalization. Additionally, using the Gain Ratio showed how normalizing for feature cardinality prevents bias toward categorical features with many unique values. Finally, the use of a separate 20% pruning set and a 5×2 cross validation protocol taught us the importance of strict data separation and statistical robustness to avoid data leakage and unreliable performance metrics.

Appendix B. Delineation of Work

- **Isaac Wilkey:** Responsible for the implementation of the core machine learning algorithms, specifically developing the decision tree growth logic using Gain Ratio for classification and Mean Squared Error for regression, as well as the reduced error pruning system. Additionally, I authored the Abstract, the Algorithms Implemented section, and the Conclusion of the final report.
- **Riley Smith:** Responsible for the implementation of the preprocessing and evaluation algorithms. In the report, I authored the Problem Statement and Hypothesis, Experimental Approach and Project Design, Experimental Results, and Analysis and Discussion sections.